Garbage Classification

Problem Statement

Background: Garbage classification is an important task in waste management and environmental conservation. Proper classification and disposal of different types of garbage play a vital role in minimizing pollution, promoting recycling, and ensuring sustainable waste management practices. With the increasing concern for environmental sustainability, the accurate classification of garbage has become crucial.

Dataset Information: The dataset used in this project is the "Garbage Classification" dataset. It consists of images of different types of garbage items, classified into various categories. The dataset contains images of items such as paper, plastic, metal, glass, cardboard, and trash. Each image is labeled with the corresponding class/category it belongs to. The dataset provides a valuable resource for training machine learning models to classify garbage items accurately.

Problem Description: The problem addressed in this project is to develop a machine learning model that can accurately classify different types of garbage items based on their images. The goal is to create a model that can classify garbage items into their respective categories, such as paper, plastic, metal, glass, cardboard, and trash. By accurately classifying garbage items, the model can assist in waste management processes, recycling initiatives, and environmental conservation efforts.

Objectives:

Build a machine learning model that can classify garbage items into their respective categories based on their images.

Achieve high accuracy and robustness in garbage classification to support effective waste management practices.

Provide a tool that can assist in automating the garbage classification process, reducing manual efforts, and promoting efficient recycling and waste disposal.

Dataset Description:

Dataset Name: Garbage Classification

Source: "Garbage Classification" dataset

Number of Classes: Multiple (paper, plastic, metal, glass, cardboard, trash)

Image Size: Resized to 256x256 pixels

Data Split: Training set, Validation set, Test set

Approach: The project follows a machine learning approach to train a model for garbage classification based on image data. The steps involved in the project include:

1. Dataset Preparation: The dataset is obtained, which contains images of different garbage items labeled with their respective categories.
2. Data Transformation: The images in the dataset are transformed to a consistent size (256x256 pixels) and converted into tensor format suitable for training machine learning models.
3. Model Selection: A pre-trained model (ResNet50) is chosen as the base model for garbage classification.
4. Model Customization: The last layer of the pre-trained model is replaced with a new fully connected layer to match the number of classes in the garbage classification task.
5. Training the Model: The model is trained on the training dataset using the training loop, which involves forward pass, loss calculation, backpropagation, and optimization.
6. Model Evaluation: The trained model is evaluated on the validation dataset to assess its performance in terms of accuracy and loss.
7. Model Fine-tuning: The model is fine-tuned by adjusting hyperparameters, such as learning rate and optimizer, to improve its performance.
8. Model Testing: The final trained model is tested on the test dataset to measure its accuracy and generalization capabilities.
9. Prediction on External Images: The trained model is used to predict the classes of external images of garbage items to demonstrate its practical application.

Framework

Importing Libraries: Begin by importing the necessary libraries, including torch, torchvision, and os. These libraries are required for working with deep learning frameworks, handling image datasets, and managing files and directories.

Loading and Exploring the Dataset: Load the Garbage Classification dataset and print the classes present in the dataset. This step helps you understand the different categories of garbage items that need to be classified.

Data Transformations: Define the transformations to be applied to the dataset. In this code, the transformations include resizing the images to a fixed size (256x256 pixels) and converting them to tensors.

Create a Helper Function to Visualize Images: Create a function that takes an image and its corresponding label as input and displays the image along with its class/category. This function helps you visualize the images in the dataset and verify their labels.

Loading and Splitting the Data: Split the dataset into training, validation, and test sets using the random_split function from the torch.utils.data module. This step ensures that you have separate subsets of data for training, validating, and evaluating the model.

Create DataLoaders: Create DataLoader objects for the training and validation sets. DataLoaders are used to load the data in batches during the training process. Specify the batch size, shuffle the training data, and set the number of worker processes for parallel data loading.

Model Base Creation: Create a base model for garbage classification. In this code, a ResNet model is used as the base model. Initialize the ResNet model and replace the last fully connected layer with a new layer that matches the number of classes in the dataset.

Define Model Evaluation Metrics: Define the evaluation metrics for the model, including an accuracy function to calculate the accuracy of predictions.

Define the ImageClassificationBase Class: Create a custom class that inherits from the nn.Module class. This class defines the training and validation steps for the model, including forward pass, loss calculation, and accuracy calculation.

Create the ResNet Model: Instantiate an object of the ResNet class, which represents the complete model for garbage classification.

GPU Configuration: Check if a GPU is available and set the device accordingly. Move the model and data to the chosen device for faster computations.

Training the Model: Define a function for model training. This function takes the number of epochs, learning rate, model, training DataLoader, validation DataLoader, and optimizer function as input. It performs the training loop, including forward and backward passes, optimization, and model evaluation. The training history is recorded for analysis.

Model Evaluation: Create a function to evaluate the model's performance on the validation dataset. This function calculates the loss and accuracy of the model predictions on the validation set.

Fit the Model: Call the fit function with the desired number of epochs, learning rate, model, training DataLoader, validation DataLoader, and optimizer function. This step trains the model and records the training history.

Visualize Training History: Plot the accuracy and loss curves over the epochs using the recorded training history. These curves provide insights into the model's training progress and performance.

Visualizing Predictions: Define a function to predict the class of an image using the trained model. This function takes an image, preprocesses it, and predicts the corresponding class/category. It also displays the image and the predicted class.

Predict on External Images: Download external images of garbage items and use the trained model to predict their classes. Display the images and the predicted classes to demonstrate the model's application on unseen data.

This outline provides a step-by-step guide to implement the Garbage Classification project using the provided code. By following this outline, you can understand the flow of the code and execute each step to train a model for accurate garbage classification.

Code Explanation

Importing Libraries:

The code begins by importing the necessary libraries: torch, torchvision, and os. These libraries provide the tools and functions required for working with deep learning frameworks, handling image datasets, and managing files and directories.

Loading and Exploring the Dataset:

- The code defines the data_dir variable, which contains the path to the dataset directory.
- It uses the os.listdir() function to retrieve a list of classes present in the dataset. Each class represents a category of garbage items.
- The list of classes is printed to the console using print(classes). This step helps us understand the different categories of garbage items that need to be classified.

Transformations:

- The code defines the transformations variable using the torchvision.transforms.Compose() function. This allows us to chain multiple image transformations together.
- In this case, two transformations are applied: resizing the images to a fixed size of 256x256 pixels and converting them to tensors. This ensures that the images are in a suitable format for training the model.

Helper Function to Visualize Images:

- The code defines a function named show_sample() that takes an image and its corresponding label as input.
- It uses print() to display the label and class number of the image.
- Then, it uses plt.imshow() to display the image.
- This function helps us visualize the images in the dataset and verify their labels.

Loading and Splitting the Data:

- The code defines the random_seed variable and sets it to 42. This ensures reproducibility of random operations.

- It uses the torch.manual_seed() function to set the random seed for PyTorch operations.
- The dataset is split into training, validation, and test sets using the random_split() function from torch.utils.data. The sizes of the splits are specified as [1593, 176, 758].
- The lengths of the training, validation, and test sets are printed to the console using len().

Create DataLoaders:

- The code imports the DataLoader class from torch.utils.data.dataloader.
- It defines the batch_size variable, which determines the number of samples processed in each batch during training and validation.
- Two DataLoader objects are created: train_dl and val_dl. These loaders are used to load the training and validation data in batches during the training process.
- The loaders shuffle the training data, specify the number of worker processes for parallel data loading, and set the pin_memory option to improve data transfer efficiency.

Model Base Creation:

- The code defines the accuracy() function, which calculates the accuracy of model predictions.
- Next, it defines the ImageClassificationBase class, which is a custom class inheriting from nn.Module.
- This class provides the base functionality for the image classification model, including training and validation steps.
- The training step calculates the loss using the cross-entropy loss function.
- The validation step calculates the loss and accuracy of the model's predictions on the validation set.
- The validation epoch end function combines the losses and accuracies from all validation batches to provide an overall metric for the epoch.
- The epoch end function prints the training and validation losses and accuracy for each epoch.

Create the ResNet Model:

- The code defines the ResNet class, which inherits from ImageClassificationBase.

- Inside the ResNet class, the ResNet-50 model is loaded using models.resnet50(pretrained=True).
- The last layer of the model is replaced with a linear layer that outputs the number of classes in the dataset.
- The forward function applies the sigmoid activation function to the output of the model and returns the result.

Porting to GPU:

- The code defines two functions: get_default_device() and to_device(), to handle device placement for computations (CPU or GPU).
- The get_default_device() function checks if a GPU is available and returns the appropriate device.
- The to_device() function moves tensors or lists of tensors to the specified device.
- The DeviceDataLoader class wraps a DataLoader object and moves the data to the specified device when iterating over batches.

Model Training:

- The code defines the evaluate() function, which evaluates the model on the validation set and returns the validation loss and accuracy.
- The fit() function trains the model for a specified number of epochs.
- Inside the fit() function, an optimizer is defined based on the chosen optimization function (Adam in this case).
- The training loop consists of iterating over batches, calculating the loss, performing backpropagation, and updating the model parameters.
- After each epoch, the model is evaluated on the validation set using the evaluate() function, and the training and validation metrics are printed.

Model Evaluation:

- The code uses the evaluate() function to evaluate the model on the validation set before training.
- The accuracy and loss of the model on the validation set are printed to the console.

Visualization of Training Progress:

- The code defines two helper functions: plot_accuracies() and plot_losses() to visualize the training progress.
- plot_accuracies() plots the accuracy values achieved during each epoch.
- plot_losses() plots the training and validation losses for each epoch.
- These visualizations help track the model's performance and identify any overfitting or underfitting issues.

Predicting External Images:

- The code demonstrates how to predict classes for external images.
- It provides a function predict_external_image() that takes the image file name as input.
- The image is loaded using PIL.Image.open() and transformed using the defined transformations.
- The transformed image is passed to the predict_image() function to get the predicted class label.
- Finally, the image and predicted label are displayed using plt.imshow() and print().

Future Work

The Garbage Classification project you have implemented is a great starting point, and there are several avenues for future work and improvements. Here is a step-by-step guide to implementing these future enhancements:

1. Dataset Expansion:

Collect more data: To improve the model's performance, you can consider expanding the dataset by collecting additional images of garbage items from various sources. This will help the model learn a wider range of features and improve its ability to generalize to new examples.

2. Data Augmentation:

Apply data augmentation techniques: Data augmentation involves applying various transformations to the existing dataset, such as rotation, scaling, and flipping. This artificially increases the dataset size and introduces more variation, making the model more robust to different types of garbage items and their orientations.

3. Transfer Learning:

Explore pre-trained models: Instead of training the model from scratch, you can leverage pre-trained models that have been trained on large-scale image datasets (e.g., ImageNet). These models have already learned generic features and can be fine-tuned on your specific garbage classification task. This approach can potentially improve the model's performance with less training data.

4. Model Architectures:

Experiment with different architectures: You can explore other popular deep learning architectures, such as DenseNet, InceptionNet, or EfficientNet, to see if they perform better than the ResNet50 model you used. Each architecture has its own strengths and may be more suitable for certain types of image classification tasks.

5. Hyperparameter Tuning:

Optimize hyperparameters: Hyperparameters, such as learning rate, batch size, and optimizer choice, greatly impact the model's performance. You can use techniques like

grid search or random search to systematically explore different combinations of hyperparameters and select the ones that yield the best performance.

6. Regularization Techniques:

Apply regularization techniques: Regularization methods, like dropout or weight decay, can help prevent overfitting and improve the model's generalization ability. You can experiment with different regularization techniques to find the optimal balance between model complexity and generalization performance.

7. Ensemble Learning:

Build an ensemble: Ensemble learning involves combining the predictions of multiple models to make final predictions. You can train multiple models with different architectures or hyperparameters and average their predictions to achieve better performance. Ensemble methods often result in improved accuracy and robustness.

8. Deployment and User Interface:

Create a user-friendly interface: Once you have a trained model, you can develop a user interface that allows users to upload images and classify them using the model. This can be done using web frameworks like Flask or Django, enabling users to interact with your model easily.

Step-by-Step Implementation Guide:

1. Expand the dataset by collecting more images of garbage items from various sources.
2. Apply data augmentation techniques to artificially increase the dataset size and introduce more variation.
3. Explore pre-trained models like DenseNet, InceptionNet, or EfficientNet and fine-tune them for your garbage classification task.
4. Optimize hyperparameters, such as learning rate, batch size, and optimizer choice, using techniques like grid search or random search.
5. Apply regularization techniques like dropout or weight decay to prevent overfitting.
6. Consider building an ensemble of models by training multiple models with different architectures or hyperparameters.

7. Develop a user-friendly interface using web frameworks like Flask or Django to allow users to interact with your trained model.
8. Continuously monitor and evaluate the performance of the model and iterate on the steps above to further improve its accuracy and usability.

By following these steps and exploring the suggested future enhancements, you can enhance the accuracy and robustness of your garbage classification system. Remember to document your progress and evaluate the model's performance at each stage to ensure continuous improvement.

Concept Explanation

Concept Explanation: The Marvelous Trash Classifier Algorithm

Imagine you're on a mission to save the world from garbage chaos! You encounter a heap of random trash items and want to classify them automatically. How would you do it? Well, our algorithm, the Marvelous Trash Classifier, comes to the rescue!

The Marvelous Trash Classifier Algorithm is like having a superhero with extraordinary vision that can instantly identify and categorize different types of garbage items. It's powered by the incredible ResNet50 model, which is trained on a large dataset of labeled garbage images.

Here's how the algorithm works in a friendly and funny manner:

Training with Superpowers: Our algorithm undergoes a rigorous training process where it learns to recognize the unique features of various garbage items. It's like attending a superhero training camp, but instead of honing physical skills, the algorithm learns to differentiate between a banana peel and an old shoe.

The Garbage Avengers Assemble: Once trained, the algorithm becomes an essential member of the Garbage Avengers team. Just like how Iron Man, Captain America, and Thor join forces, our algorithm collaborates with other components to fight the battle against garbage chaos.

Image Transformation Magic: To prepare the garbage items for classification, the algorithm applies some magical transformations. It resizes each item to a standard size (256x256 pixels) and converts it into a tensor. It's like using a shrink ray to make all garbage items fit in the same-sized container.

Summoning the Trash Dataset: Our algorithm is armed with a special dataset called the Trash Dataset. It contains a wide range of labeled images, representing different types of garbage items. It's like having an encyclopedia of trash knowledge at its disposal.

Training and Validation: The algorithm trains its superhero powers by dividing the Trash Dataset into training and validation sets. It learns from the training set, constantly fine-tuning its garbage classification skills. It's like a superhero attending training sessions and occasionally taking exams to gauge their progress.

The Battle of Accuracy: During training, the algorithm battles against the arch-nemesis called "Loss." It constantly measures how well it predicts the correct garbage class for each item. The lower the loss, the better the algorithm becomes at accurate classification. It's like battling an evil villain who tries to confuse the algorithm.

Validation, the Superhero Assessment: After each training session, the algorithm undergoes a validation test. It faces unseen garbage items from the validation set and showcases its superpowers. It measures its accuracy, which is like a superhero getting graded on their heroic acts.

The Heroic Epochs: The algorithm goes through multiple iterations called "epochs," where it repeats the training and validation steps. With each epoch, the algorithm becomes more skilled at distinguishing between a soda can and a milk carton. It's like a superhero getting stronger and more experienced over time.

The Final Showdown: Testing Once fully trained, the algorithm is ready for the ultimate showdown. It faces the ultimate challenge—garbage items it has never seen before. It predicts the correct class for each item and showcases its true superpowers.

Celebrating the Trash Sorting Victory: When the algorithm classifies garbage items with impressive accuracy, it's time to celebrate the victory over garbage chaos. Confetti falls, music plays, and the algorithm basks in the glory of its successful classification achievements.

So there you have it, the Marvelous Trash Classifier Algorithm explained in a fun and engaging way! It's like having a superhero with extraordinary trash-sorting powers. With its training, validation, and testing adventures, it fights against the chaos of garbage, saving the world, one banana peel at a time.

Exercise Questions

Question: Can you explain the purpose of the fit() function in the code and how it is used for training the model?

Answer: The fit() function is responsible for training the model. It takes several parameters, including the number of epochs (iterations), learning rate (lr), the model itself, and the training and validation data loaders. Inside the function, it performs the training and validation steps for the specified number of epochs. It uses the provided optimizer function (in this case, Adam) to optimize the model's parameters based on the computed loss. The training losses are collected and used to calculate the average training loss per epoch. The validation step is performed after each epoch to evaluate the model's performance on unseen data. The training loss, validation loss, and validation accuracy are displayed for each epoch.

Question: In the code, there is a function called evaluate(). What is the purpose of this function, and how is it used during the training process?

Answer: The evaluate() function is used to evaluate the model's performance on the validation set during the training process. It takes the model and the validation data loader as inputs. Inside the function, it sets the model to evaluation mode using model.eval(), then iterates through the validation data loader to calculate the loss and accuracy for each batch. It returns the average validation loss and accuracy across all batches. This function is called within the fit() function after each epoch to obtain the validation metrics.

Question: What is the role of the ResNet class in the code, and how does it relate to the overall model architecture?

Answer: The ResNet class is a custom class that extends the ImageClassificationBase class, which itself extends the nn.Module class in PyTorch. The ResNet class defines the architecture of the model based on the ResNet50 pre-trained model. It replaces the last fully connected layer of the pre-trained model with a new linear layer to match the number of classes in the dataset. This custom architecture is used to create an instance of the model for garbage classification. The forward() method in the ResNet class defines the forward pass of the model, where input images are passed through the network to generate predictions.

Question: Explain the purpose of the DeviceDataLoader class and how it enables efficient GPU usage during training.

Answer: The DeviceDataLoader class is a wrapper around the regular data loader (DataLoader) that helps move the data to the appropriate device (GPU or CPU). It takes a data loader and the target device as inputs. When iterated, the DeviceDataLoader converts the data (batches) to the specified device using the to_device() function. By moving the data to the GPU, it allows the model to utilize the GPU's parallel processing capabilities, leading to faster training. It also sets the non_blocking parameter to True, which allows the data transfers to be asynchronous, further improving performance.

Question: How does the algorithm handle predicting classes for external images that were not part of the training or validation sets?

Answer: To predict classes for external images, the algorithm uses the predict_image() function. It takes an image and the trained model as inputs. First, the image is transformed and converted into a tensor using the same transformations applied during training. Then, the tensor is passed through the model to obtain the predicted probabilities for each class. The class with the highest probability is considered the predicted class for the image. This approach allows the model to make predictions on unseen images by leveraging its learned knowledge from the training data.